

Process Scheduling

(~Available for Android)

From Modern Operating Systems

A. S. Tanenbaum

Second edition

Linux

- As of Dec 2015
 - Webservers: W3Cook reports that 96.5% of web servers run Linux (1.5% run Windows)
 - Desktops/laptops: 1.5% use Linux
 - Mobile devices: Android (based on Linux kernel) is used in 80% of all mobile devices
 - Platform of choice for film industry
- Linux is free and open-source
- Please note that Linux uses the term “task”

Linux Scheduling

- Linux has these scheduling classes:
 - Real-time (RT) classes
 - Completely fair scheduler (CFS) class
- Tasks in RT have higher precedence than tasks in the CFS

Linux Scheduler

- 140 priority levels:
 - RT: There are 100 priority lists ranging from 0 to 99.
 - User tasks: 101-140 (nice values -20 to 19)
- Within RT there are two scheduling policies
 - SCHED_RR: round-robin
 - SCHED_FIFO: FIFO
- Round-robin has precedence over FIFO

Scheduler Policies

- Scheduling policy is set by `sched_setscheduler()`
- Scheduling policies:
 - `SCHED_FIFO`: for special time-critical applications
 - `SCHED_RR` round robin scheduling
 - `SCHED_BATCH` for CPU-intensive tasks
 - `SCHED_IDLE` for low prioritized tasks
 - `SCHED_OTHER`: default

Processor scheduling

- Processor scheduling policy
 - Decides which process runs at given time
 - Different schedulers will have different goals
 - Maximize throughput
 - Minimize latency
 - Prevent indefinite postponement
 - Complete process by given deadline
 - Maximize processor utilization

Preemptive vs. Nonpreemptive Scheduling

- Preemptive processes
 - Can be removed from their current processor
 - Can lead to improved response times
 - Important for interactive environments
 - Preempted processes remain in memory
- Nonpreemptive processes
 - Run until completion or until they yield control of a processor
 - Unimportant processes can block important ones indefinitely

Scheduling Algorithm Goals

All systems

Fairness - giving each process a fair share of the CPU

Policy enforcement - seeing that stated policy is carried out

Balance - keeping all parts of the system busy

Batch systems

Throughput - maximize jobs per hour

Turnaround time - minimize time between submission and termination

CPU utilization - keep the CPU busy all the time

Interactive systems

Response time - respond to requests quickly

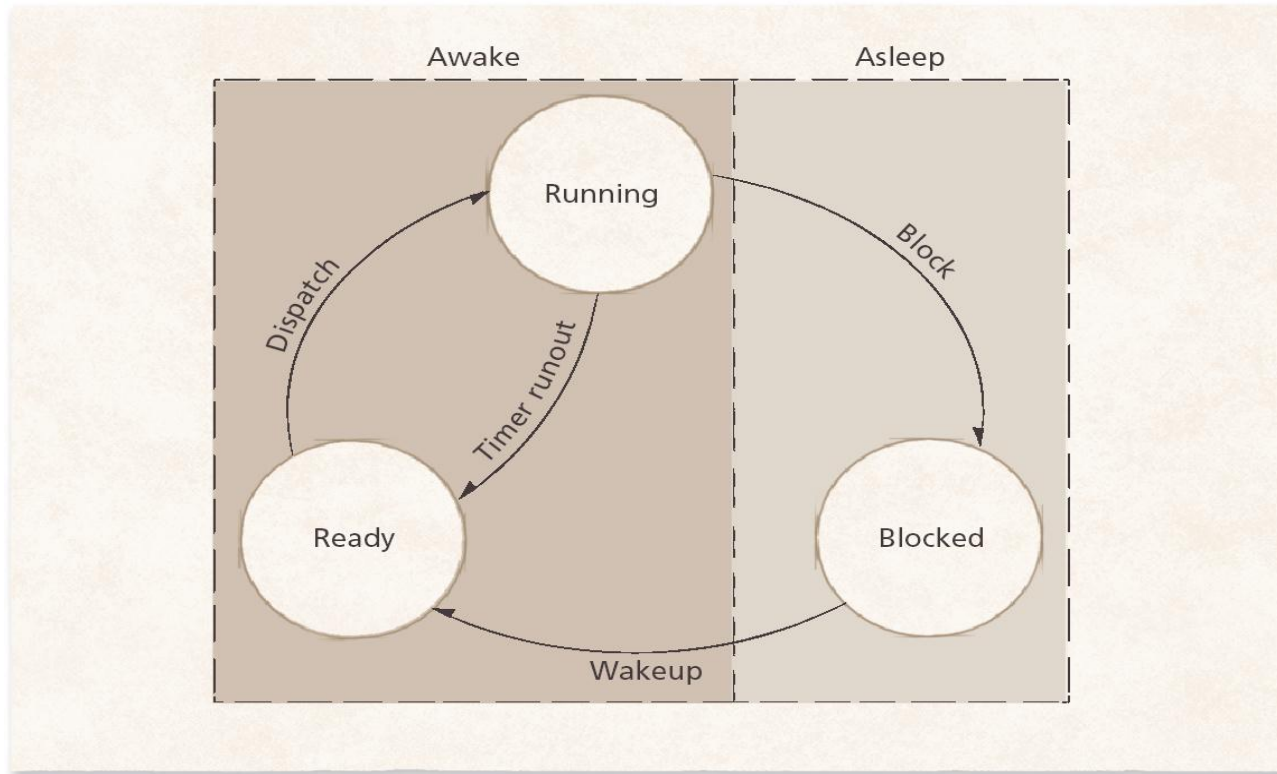
Proportionality - meet users' expectations

Real-time systems

Meeting deadlines - avoid losing data

Predictability - avoid quality degradation in multimedia systems

Process States and State Transitions



First-In-First-Out (FIFO) Scheduling

- FIFO scheduling
 - Simplest scheme
 - Processes dispatched according to arrival time
 - Nonpreemptible
 - Rarely used as primary scheduling algorithm

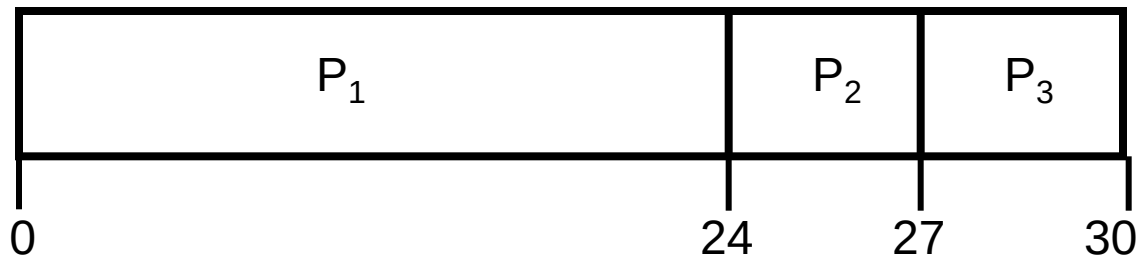
FIFO

Example:

<u>Process</u>	<u>Burst</u>
P1	24
P2	3
P3	3

At time 0 the arriving order is: P1 , P2 , P3

Gantt's diagram is:



Waiting time(time spent in Queue): P1= 0; P2= 24; P3= 27

Average waiting time: $(0 + 24 + 27)/3 = 17$

Average turnaround time: $(24+27+30)/3 = 27$

Shortest-Job-First (SJF)

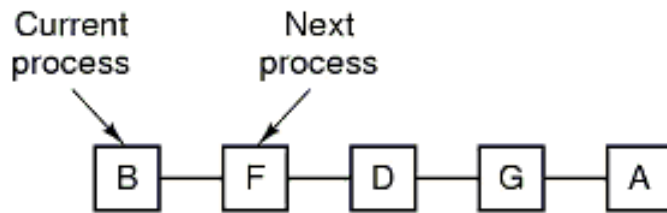
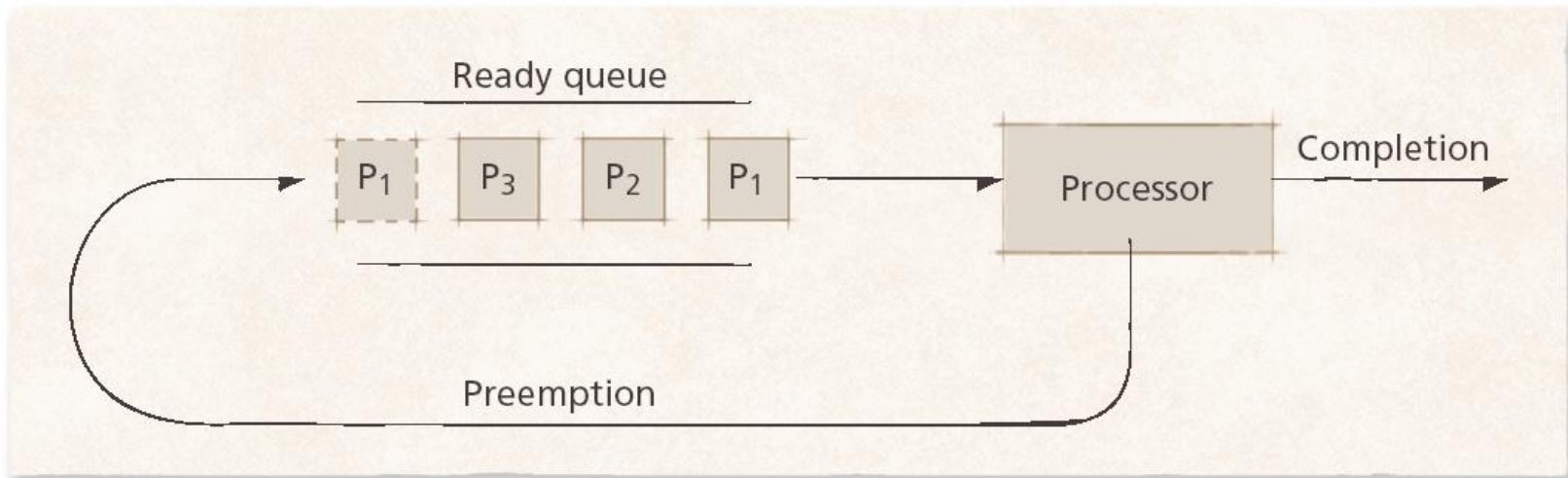
Scheduling

- Scheduler selects process with smallest time to finish
 - Lower average wait time than FIFO
 - Reduces the number of waiting processes
 - Potentially large variance in wait times
 - Nonpreemptive
 - Results in slow response times to arriving interactive requests
 - Relies on estimates of time-to-completion
 - Can be inaccurate or falsified
 - Unsuitable for use in modern interactive systems

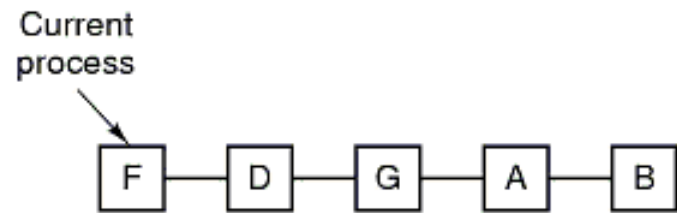
Round-Robin (RR) Scheduling

- Round-robin scheduling
 - Based on FIFO
 - Processes run only for a limited amount of time called a time slice or quantum
 - Preemptible
 - Requires the system to maintain several processes in memory to minimize overhead
 - Often used as part of more complex algorithms

RR



(a)



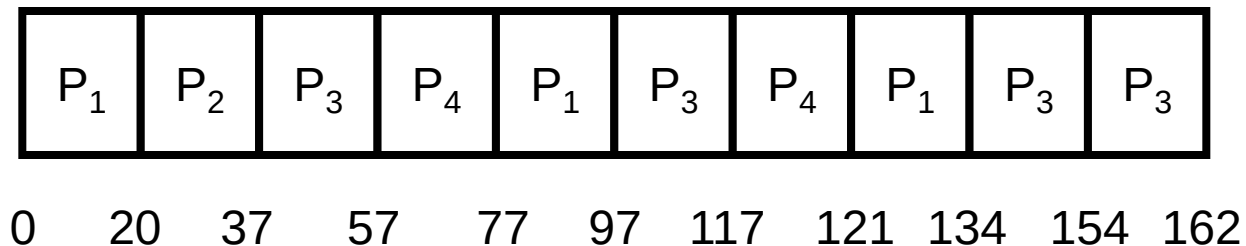
(b)

- Round Robin Scheduling

- list of runnable processes
- list of runnable processes after B uses up its quantum

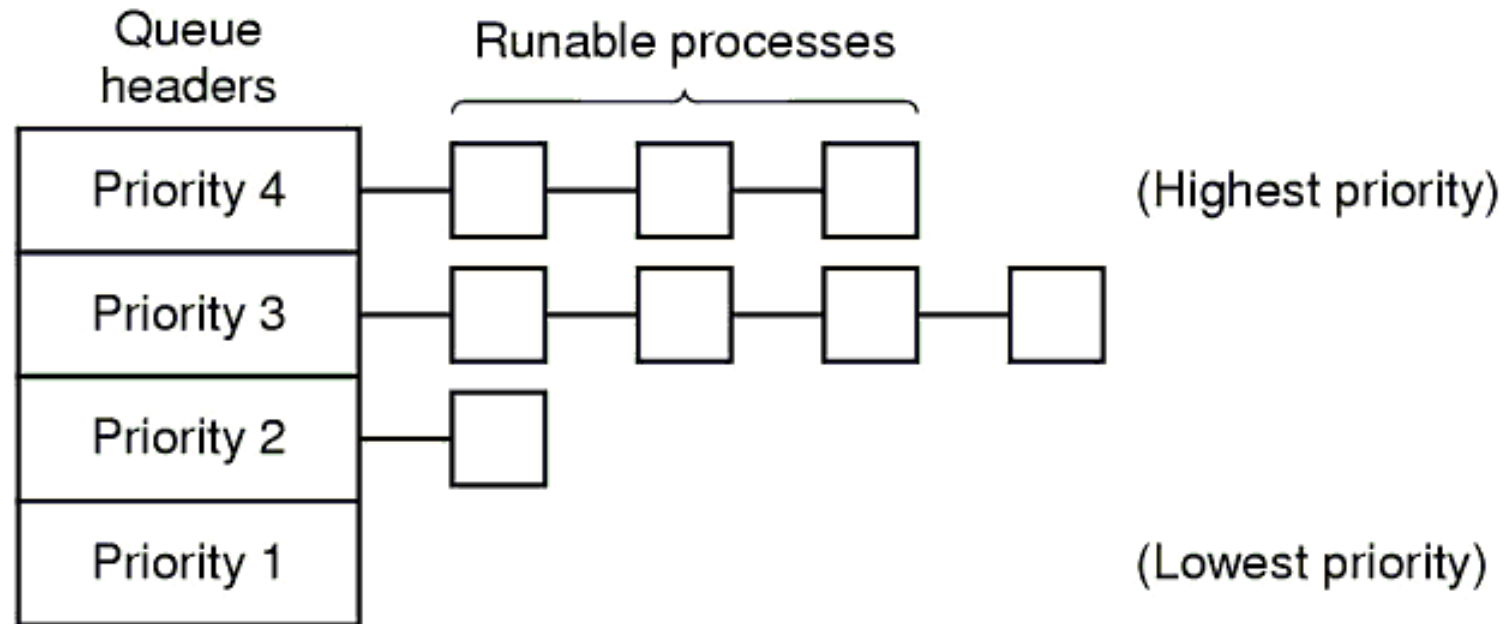
Example: RR Quantum = 20

<u>Process</u>	<u>Cycle</u>
P_1	53
P_2	17
P_3	68
P_4	24



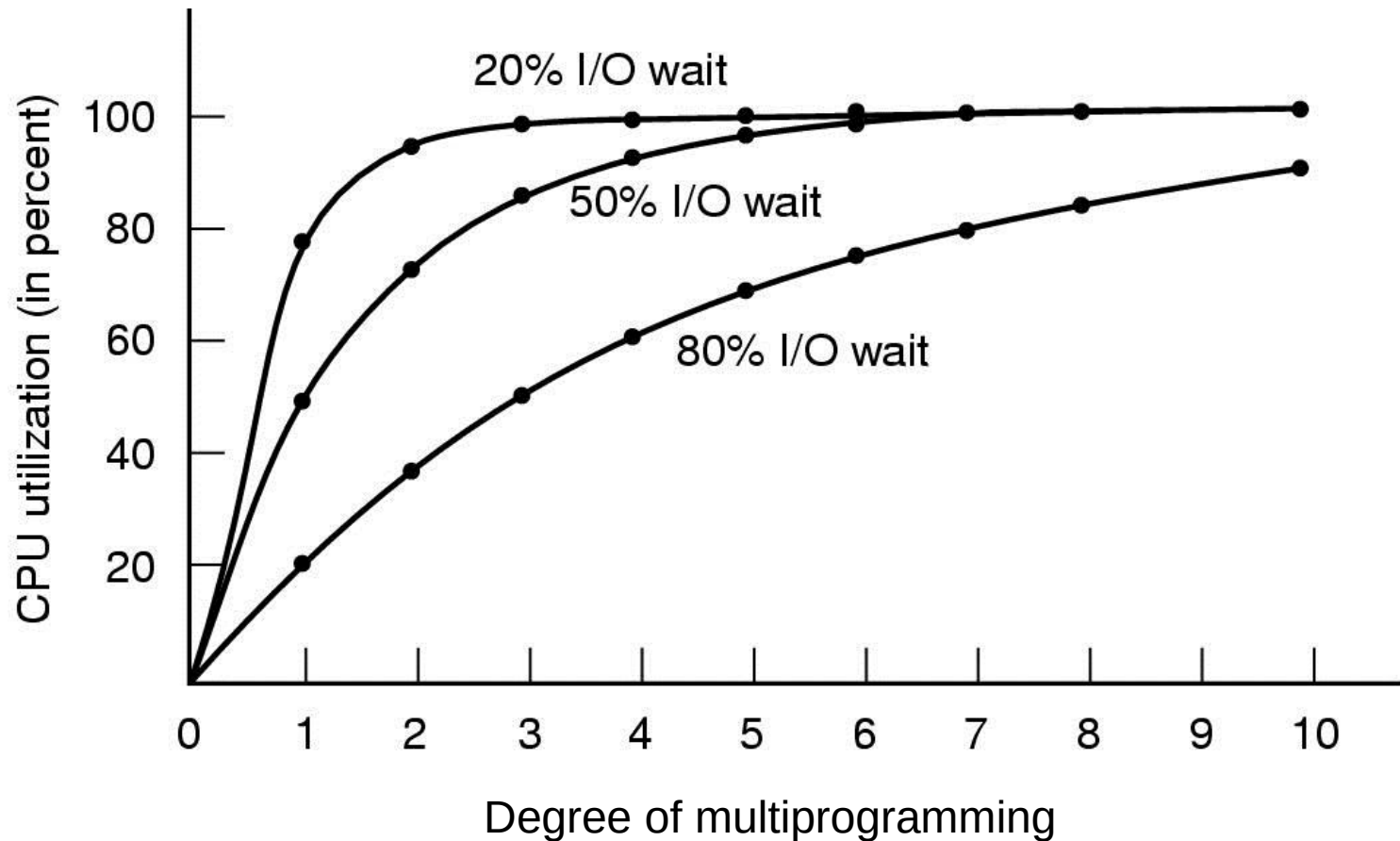
- Normally,
 - turnaround higher than SJF
 - Waiting time is better

Scheduling in Interactive Systems



A scheduling algorithm with four priority classes

Modeling Multiprogramming



CPU utilization as a function of number of processes in memory

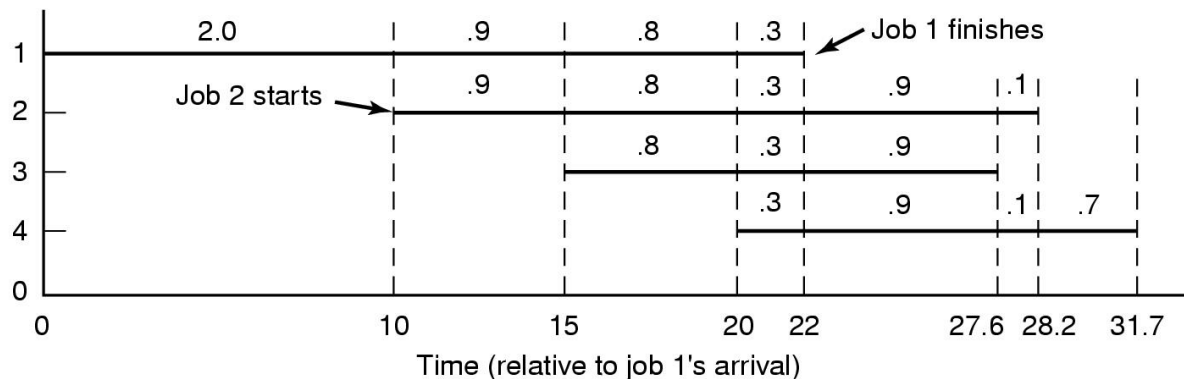
Analysis of Multiprogramming System Performance

Job	Arrival time	CPU minutes needed
1	10:00	4
2	10:10	3
3	10:15	2
4	10:20	2

(a)

	# Processes			
	1	2	3	4
CPU idle	.80	.64	.51	.41
CPU busy	.20	.36	.49	.59
CPU/process	.20	.18	.16	.15

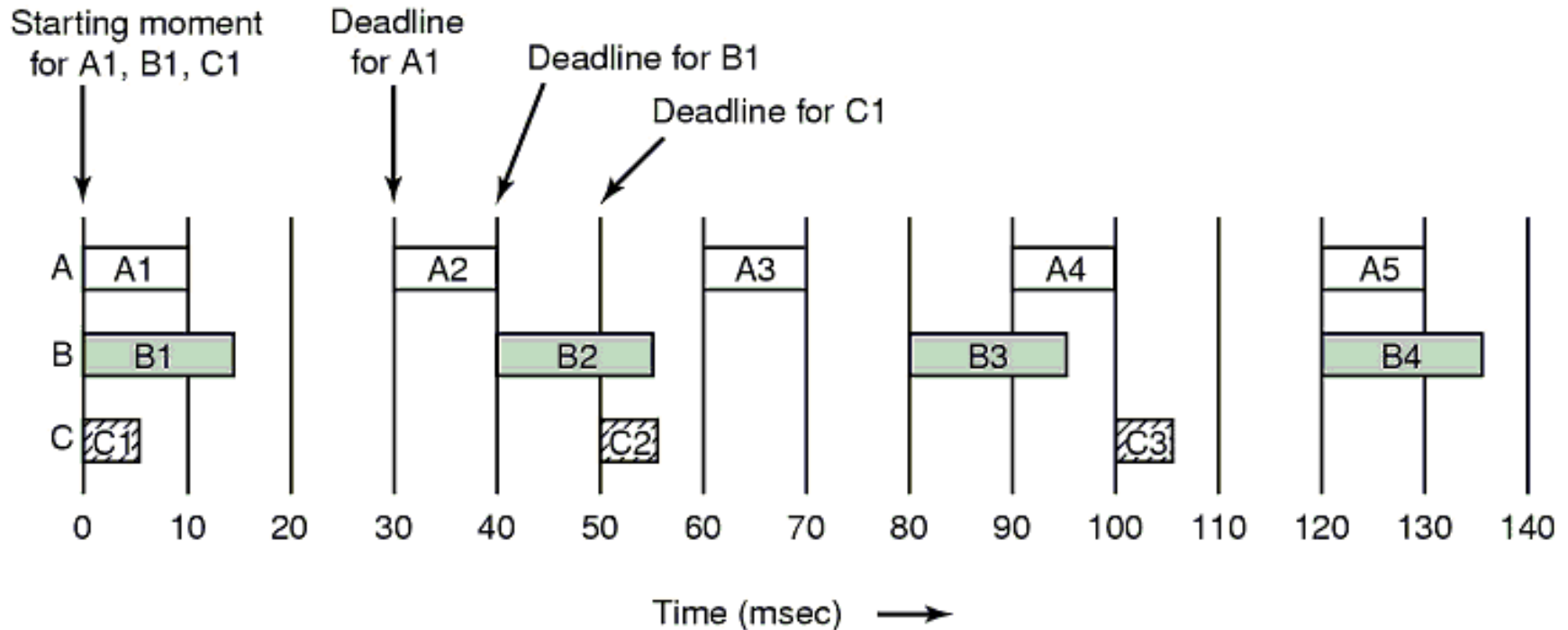
(b)



(c)

- Arrival and work requirements of 4 jobs
- CPU utilization for 1 – 4 jobs with 80% I/O wait
- Sequence of events as jobs arrive and finish
 - note numbers show amount of CPU time jobs get in each interval

Real-time Process Scheduling



- Periodic processes displaying a movie
- Frame rates and processing requirements may be different for each movie

Scheduling in Real-Time Systems

Schedulable real-time system

- Given
 - m periodic events
 - event i occurs within period P_i and requires C_i seconds
- Then the load can only be handled if

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1$$

Rate Monotonic Scheduling

Used for processes which meet these conditions

1. Each periodic process must complete within its period
2. No process dependent on any other process
3. Each process needs same CPU time each burst
4. Any nonperiodic processes have no deadlines
5. Process preemption occurs instantaneously, no overhead

Rate Monotonic Scheduling

- Each process will have a fixed priority equal to the frequency of occurrence of its triggering event
- For example,
 - a process that must run every 30 msec (33 times/sec) gets priority 33
 - a process that must run every 40 msec (25 times/sec) gets priority 25
 - a process that must run every 50 msec (20 times/sec) gets priority 20
- At run time, the scheduler always run the highest priority ready process, preempting the running process if need be

Rate Monotonic Scheduling

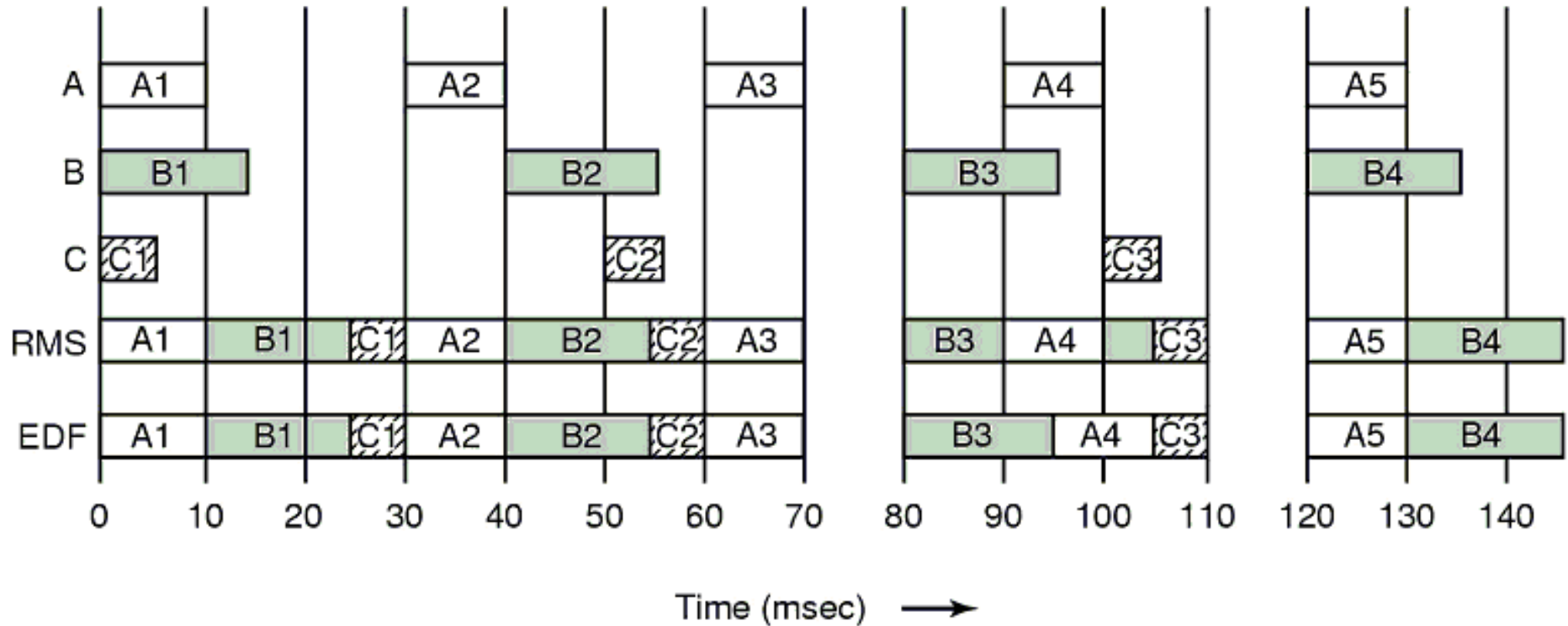
- Optimal among the class of static scheduling algorithms
- Condition for success but not for failure

$$\sum_{i=1}^m \frac{C_i}{P_i} \leq m(2^{1/m} - 1)$$

Dynamic real-time scheduling

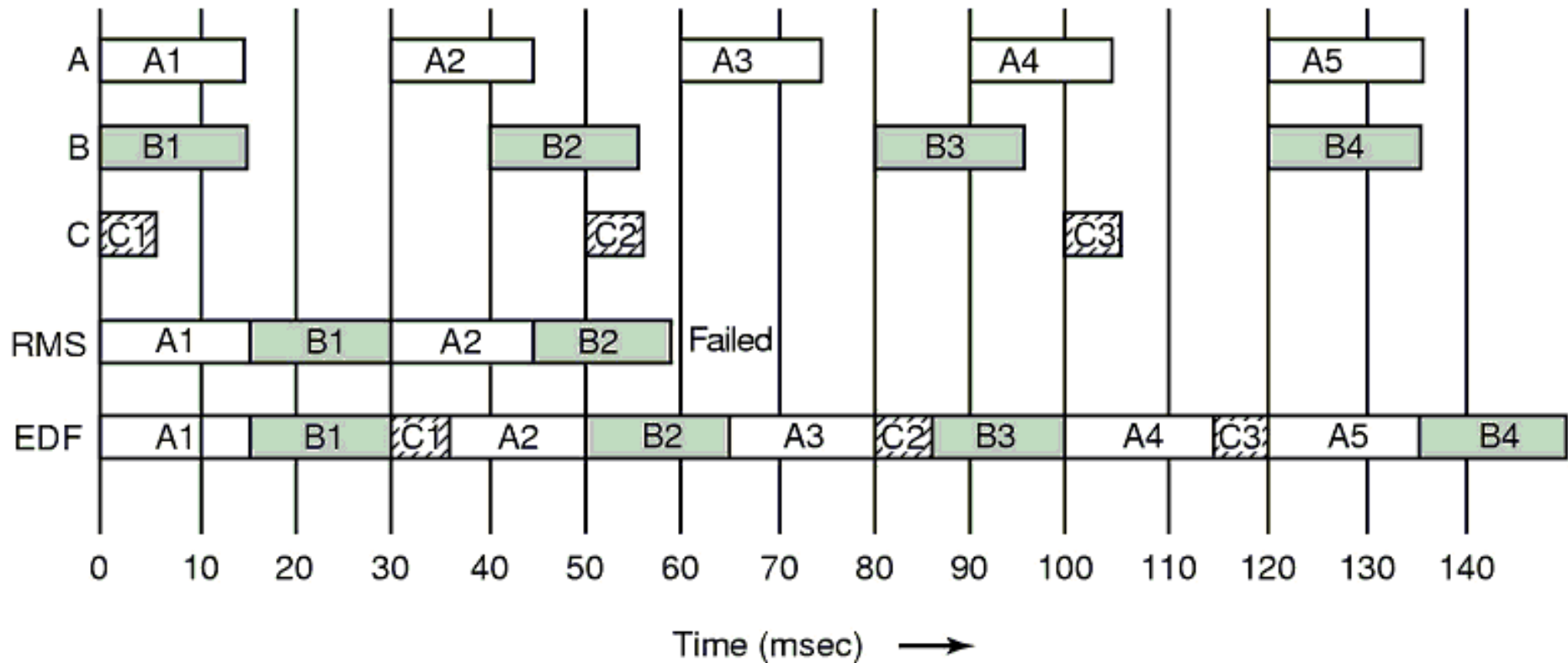
- Dynamic real-time scheduling
 - Adjusts priorities in response to changing conditions
 - Can incur significant overhead, but must ensure that the overhead does not result in increased missed deadlines
 - Priorities are usually based on processes' deadlines
 - Earliest-deadline-first (EDF)
 - Preemptive, always dispatch the process with the earliest deadline
 - Least-laxity-first (LLF)
 - Similar to EDF, but bases priority on laxity, which is based on the process's deadline and its remaining run-time-to-completion

Example



- Real Time Scheduling algorithms
 - RMS
 - EDF

Example

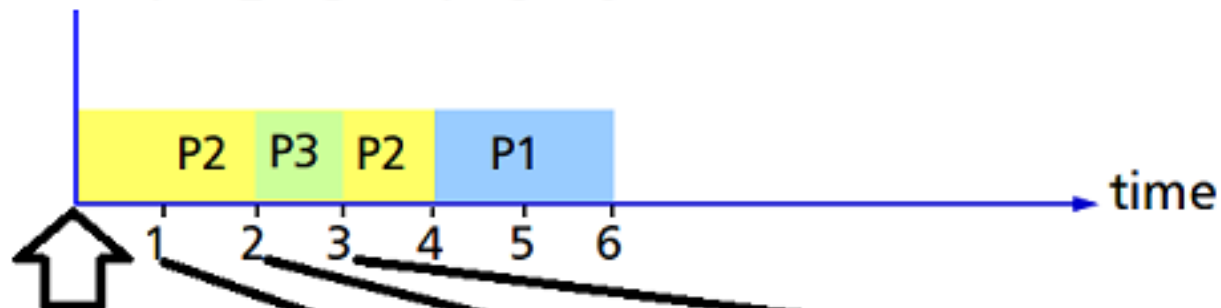
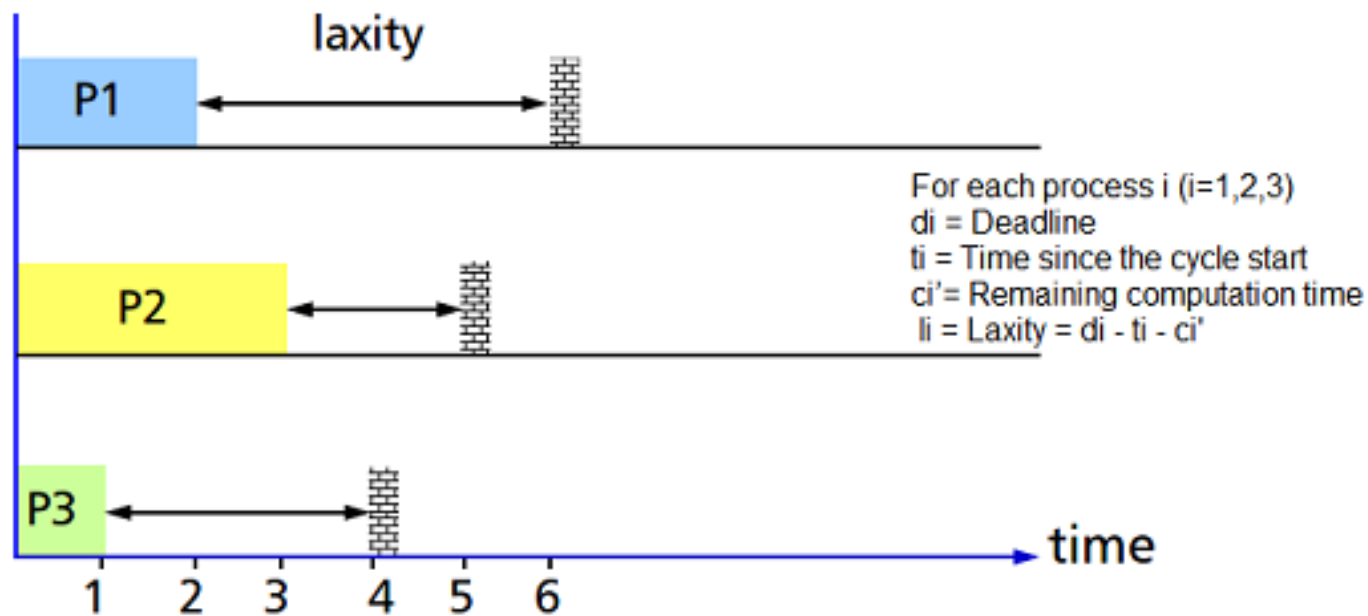


Another example of real-time scheduling with RMS and EDF

Least Laxity First (LLF)

- Dynamic scheduling algorithm
- First selects those processes that have the smallest laxity/margin or "slack time".
- Slack time is defined as the temporal difference between the deadline, the ready time and the run time.
- More formally, the *slack time* for a process is defined as: $(d - t) - c'$
 - Where d is the process deadline, t is the real time since the cycle start, and c' is the remaining computation time.

► **Example:** equal ready times



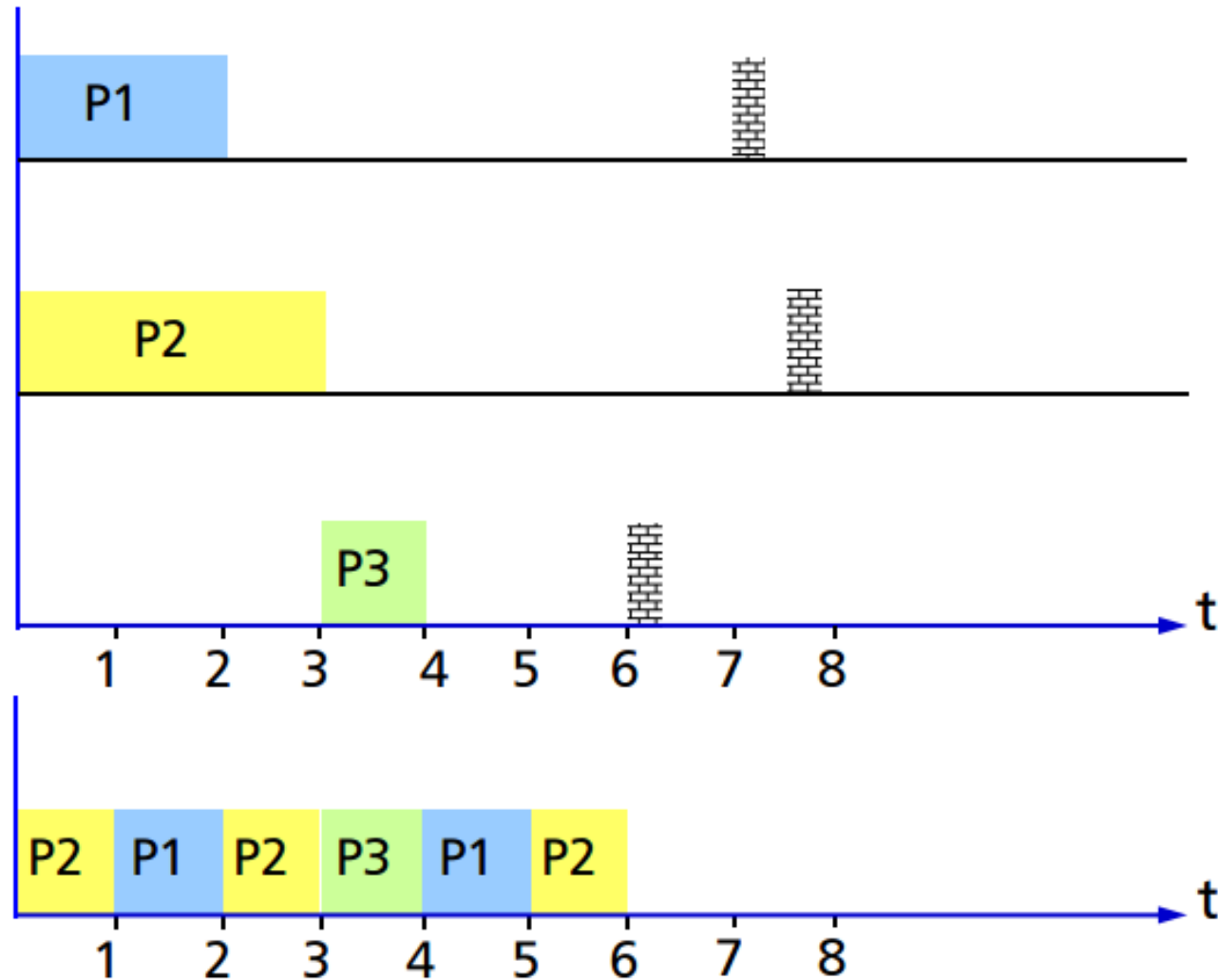
At $t=0$, P2 is selected since
 $d_1 = 6, t_1 = 0, c_1' = 2 \rightarrow l_1 = 4$
 $d_2 = 5, t_2 = 0, c_2' = 3 \rightarrow l_2 = 2$
 $d_3 = 4, t_3 = 0, c_3' = 1 \rightarrow l_3 = 3$

At $t=1$, P2 continues
 $l_1 = 3$
 $l_2 = 2$
 $l_3 = 2$

At $t=2$, P3 is selected
 $l_1 = 2$
 $l_2 = 2$
 $l_3 = 1$

At $t=3$, P3 ends
P1 or P2 could be selected
 $l_1 = 1$
 $l_2 = 1$

Example: different ready times, preemptive schedule



Completely fair scheduler (CFS)

- To maintain balance (fairness) in providing processor time to tasks.
- i.e. processes should be given a fair amount of the processor.
 - When the time for tasks is out of balance (meaning that one or more tasks are not given a fair amount of time relative to others), then those out-of-balance tasks should be given time to execute.
- based on selecting the Process to run next with the smallest virtual runtime.
- How to compute this vruntime?

Example

- Assume two tasks t1 and t2 with nice values of 0
- Assume t1 runs for 200 milliseconds and t2 runs for 100 milliseconds which is followed by a lot of other tasks
- t2 will be selected before t1 for execution

How to compute this vruntime?

- A Process has a nice value (+ or –)
- Each nice value (let's say i) is also associated with a weight $W(i)$
 - $W(i) = 1024 * \text{power}(1.25, -i)$
 - $W(0) = 1024$
- Decay Factor is given using the formula:
 - $Df(i) = W(0) / W(i)$
- A CPU has what's called a targeted latency called TL, and each process needs to have a slice of time out of this TL (denoted $Ts(i)$), hence the formula of the time slice is given by:
 - $Ts(i) = TL * W(i) / \sum W(j)$
- vruntime is initially 0 and incremented by $Ts(i)*Df(i)$ each time a process run.

CFS – Virtual Runtime

- Example: Assume a process runs for 200 milliseconds
 - Nice value is 0: vruntime will be 200 milliseconds
 - Nice value < 0 : vruntime will be less than 200 milliseconds
 - Nice value > 0 : vruntime will be greater than 200 milliseconds
- Smaller nice values results in values of vruntime that grows more slowly than higher nice values

Example

- Assume two tasks t1 and t2 with nice values of 0 and 5 respectively w_z vruntime₀ and vruntime₁ are initially zero
 - Decay factors of 1 and 3.05 respectively
 - Assume t1 ran for 200 milliseconds and t2 ran for 100 milliseconds which is followed by a lot of other tasks
 - vruntime₀ is 200 milliseconds and vruntime₁ is 305 milliseconds
- t1 will be selected before t2 for execution
- Let's say that t1 runs again for 200 milliseconds; vruntime₀ is now 400 milliseconds
- t2 will now be selected before t1 for execution

CFS - implementation

- No run queue
- CFS maintains a time-ordered red-black tree

